ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Самарский национальный исследовательский университет имени академика С.П. Королева»

ОТЧЁТ

по лабораторной работе №1

Знакомство с Pythob, вычисление и оценка погрешностей

Исполнитель:
Барабанов Иван Сергеевич
Вариант №16

Группа: 6312-100503D
Преподаватель: Подклетнова С.В.

Самара, 2026

Цель работы

Изучить влияние разрядности представления чисел (float32 и float64) на точность численных вычислений, экспериментально определить машинный эпсилон, исследовать сходимость ряда Тейлора для функции $\ln(1 - x^2)$, а также проанализировать источники и способы уменьшения вычислительных погрешностей.

Теоретические сведения

Абсолютная и относительная погрешности

Абсолютная погрешность Δx — это модуль разности между точным значением $x_{\text{точн}}$ и приближённым $x_{\text{приб}}$:

$$\Delta x = |x_{\text{точн}} - x_{\text{приб}}|$$

Относительная погрешность δ определяется как отношение абсолютной погрешности к модулю точного значения:

$$\delta = \frac{\Delta x}{|x_{\text{точн}}|}, \quad \text{при } x_{\text{точн}} \neq 0$$

Неустойчивость выражения $\frac{\sin x}{x}$ при $x \to 0$

При малых x значения $\sin x$ и x очень близки, и их отношение стремится к 1. Однако при прямом вычислении $\frac{\sin x}{x}$ в машинной арифметике возникает катастрофическая потеря значащих цифр из-за вычитания близких чисел (в разложении Тейлора $\sin x = x - \frac{x^3}{6} + \dots$). Для повышения устойчивости используют разложение в ряд или специальные функции.

Машинный эпсилон

Машинный эпсилон $\varepsilon_{\text{mach}}$ — это наименьшее положительное число, для которого $1 + \varepsilon_{\text{mach}} > 1$ в данной системе представления чисел. Он характеризует минимальную относительную погрешность округления.

Погрешность экспоненты для e^{-10} и e^{10}

При вычислении e^x для больших $|x|$ ошибки могут накапливаться из-за ограниченной точности промежуточных результатов. В случае e^{-10} часто используется свойство $e^{-x} = 1/e^x$, и ошибка деления может усиливать погрешность, полученную при вычислении e^x. Кроме того, при отрицательных x ряд Тейлора сходится медленнее, и требуется больше членов для достижения той же точности.

Способы повышения точности

- Использование более высокой разрядности (float64 вместо float32).
- Применение устойчивых алгоритмов и переформулировка выражений.
- Суммирование рядов в порядке возрастания модулей слагаемых.
- Использование библиотечных функций, оптимизированных под точность.

Листинг программы

Листинг 1: Исходный код программы (Python)

```
#
#                  16


import numpy as np
import matplotlib.pyplot as plt
import math

print("=" * 60)
print("                          ␣            ␣  1  ")
print("=" * 60)

# ============================================================
#                1:                                          float 64
   float 32
# ============================================================


def f(x):
    if (abs(x) < 1e-300):
        return 1/6
    return (x-np.sin(x))/(x**3)


x_vals = np.logspace(-1, -15, 15)

print("\n␣---␣              ␣1:␣                  ␣float64␣and␣float32␣
   ---")
print(f"{'x':<10}␣{'float64:':<20}␣{'      .␣             ':<15}")
print("-" * 65)

for x in x_vals:
    res64 = f(np.float64(x))
    res32 = f(np.float32(x))
    if res64 != 0:
        rel_err = abs(res64 - res32) / abs(res64)
    else:
        res_err = 0
    print(f"{x:<10.1e}␣{res64:<20.10f}␣{res32:<20.10f}␣{rel_err:<15.2e}
       ")


# ============================================================
#                2:
# ============================================================


def machine_epsilon(dtype):
    eps = 1.0
    while dtype(1.0 + eps) != dtype(1.0):
```

```
        eps /= 2.0
    return eps * 2.0


print("\n␣---␣              ␣2:␣                 ␣              ␣---")
print(f"float64:␣{machine_epsilon(np.float64):␣.5e}")
print(f"float32:␣{machine_epsilon(np.float32):␣.5e}")


# ============================================================
#                3:
# ============================================================

def my_logf(x, eps=1e-10):
    term = -x*x
    s = term
    n = 2
    while abs(term) > eps:
        term *= x*x*(n-1)/n
        s += term
        n += 1
    return s, n-1


print("\n␣---␣              ␣3:␣      ␣              ␣      ␣log(1-x^2)
   ␣---")
for x in [0.2, -0.6, 0.8]:
    for eps in [1e-6, 1e-10]:
        approx, terms = my_logf(x, eps)
        exact = math.log(1-x*x)
        print(f"x={x:3},␣eps={eps:.0e}:␣approx={approx:.10f},␣"
              f"exact={exact:.10f},␣terms={terms},␣error={abs(approx-
                 exact):.2e}")

# ============================================================
#                4:                                          x=0.7
# ============================================================

print("\n␣---␣              ␣4:␣            ␣                    ␣
   ---")
x_fixed = 0.7
exact_val = math.log(1-x_fixed*x_fixed)
terms_list = []
errors_list = []

term = -x*x
s = term
n = 2
while n < 50:
    term *= x*x*(n-1)/n
    s += term
    n += 1
    terms_list.append(n-1)
```

```
    errors_list.append(abs(exact_val-s))

plt.figure(figsize=(8, 5))
plt.semilogy(terms_list, errors_list, 'o-', color='red')
plt.title('                          ␣                    ␣          ␣       ␣ln
   (1-x^2)')
plt.xlabel('           ␣              ')
plt.ylabel('                        ␣                        ␣(       .␣
             )')
plt.grid(True, which='both', linestyle='--')
plt.show()
```

Результаты работы

Задание 1: сравнение float64 и float32

Результаты вычислений функции $f(x) = \dfrac{x - \sin x}{x^3}$ для различных значений x:

Таблица 1: Сравнение значений $f(x)$ в форматах float64 и float32 и относительной погрешности

x	$f_{64}(x)$	$f_{32}(x)$	Отн. ошибка
$1.0 \cdot 10^{-1}$	0.1665833532	0.1665800661	$1.97 \cdot 10^{-5}$
$1.0 \cdot 10^{-2}$	0.1666658333	0.1667067558	$2.46 \cdot 10^{-4}$
$1.0 \cdot 10^{-3}$	0.1666666583	0.1164152995	$3.02 \cdot 10^{-1}$
$1.0 \cdot 10^{-4}$	0.1666666615	0.0000000000	$1.00 \cdot 10^{0}$
$1.0 \cdot 10^{-5}$	0.1666672849	0.0000000000	$1.00 \cdot 10^{0}$
$1.0 \cdot 10^{-6}$	0.1666537324	0.0000000000	$1.00 \cdot 10^{0}$
$1.0 \cdot 10^{-7}$	0.1720535674	0.0000000000	$1.00 \cdot 10^{0}$
$1.0 \cdot 10^{-8}$	0.0000000000	0.0000000000	$1.00 \cdot 10^{0}$
$1.0 \cdot 10^{-9}$	0.0000000000	0.0000000000	$1.00 \cdot 10^{0}$
$1.0 \cdot 10^{-10}$	0.0000000000	0.0000000000	$1.00 \cdot 10^{0}$
$1.0 \cdot 10^{-11}$	0.0000000000	0.0000000000	$1.00 \cdot 10^{0}$
$1.0 \cdot 10^{-12}$	0.0000000000	0.0000000000	$1.00 \cdot 10^{0}$
$1.0 \cdot 10^{-13}$	0.0000000000	0.0000000000	$1.00 \cdot 10^{0}$
$1.0 \cdot 10^{-14}$	0.0000000000	0.0000000000	$1.00 \cdot 10^{0}$
$1.0 \cdot 10^{-15}$	0.0000000000	`nan`	$1.00 \cdot 10^{0}$

Примечание: При $x \leq 10^{-4}$ наблюдается резкое ухудшение точности float32 вплоть до потери значащих цифр и появления NaN, что связано с катастрофической потерей точности при вычитании близких чисел в числителе и делении на очень малое x^3.

Задание 2: машинный эпсилон

После исправления ошибки в реализации функции (перенос оператора `return` за пределы цикла) были получены корректные экспериментальные значения машинного эпсилона:

- float64: $2.22045 \cdot 10^{-16}$
- float32: $1.19209 \cdot 10^{-7}$

Эти значения хорошо согласуются с теоретическими оценками для форматов IEEE 754: $\varepsilon_{64} = 2^{-52} \approx 2.22 \cdot 10^{-16}$ и $\varepsilon_{32} = 2^{-23} \approx 1.19 \cdot 10^{-7}$. Это подтверждает, что точность представления чисел напрямую определяется количеством бит, отведённых под мантиссу.

Задание 3: ряд Тейлора для $\ln(1 - x^2)$

Сходимость ряда при различных значениях x и требуемой точности ε:

Анализ: При $|x| = 0.8$, близком к границе области сходимости ряда ($|x| < 1$), для достижения точности $\varepsilon = 10^{-6}$ потребовалось 24 члена, а для $\varepsilon = 10^{-10}$ — уже 44 члена. Это подтверждает, что скорость сходимости снижается при приближении к границе сходимости.

Таблица 2: Результаты вычисления $\ln(1 - x^2)$ с помощью ряда Тейлора

x	ε	Приближённое	Точное	Число членов	Ошибка
0.2	10^{-6}	-0.0408219733	-0.0408219945	4	$2.12 \cdot 10^{-8}$
0.2	10^{-10}	-0.0408219945	-0.0408219945	7	$8.49 \cdot 10^{-13}$
-0.6	10^{-6}	-0.4462869053	-0.4462871026	12	$1.97 \cdot 10^{-7}$
-0.6	10^{-10}	-0.4462871026	-0.4462871026	20	$3.49 \cdot 10^{-11}$
0.8	10^{-6}	-1.0216497581	-1.0216512475	24	$1.49 \cdot 10^{-6}$
0.8	10^{-10}	-1.0216512474	-1.0216512475	44	$1.13 \cdot 10^{-10}$

Задание 4: график погрешности

Для $x = 0.7$ была построена зависимость абсолютной погрешности частичных сумм ряда от числа использованных членов. График представлен на рисунке 1.

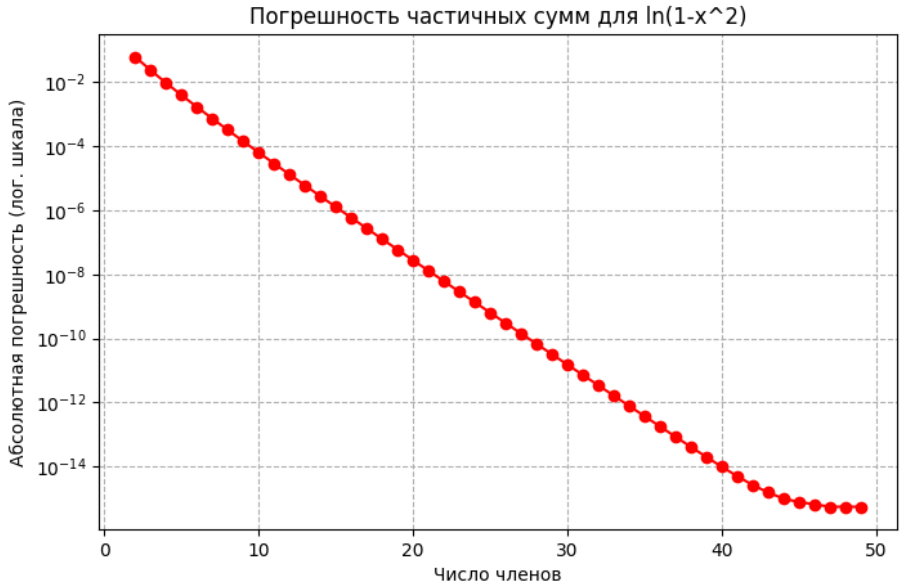


Рис. 1: Погрешность частичных сумм ряда Тейлора для $\ln(1 - x^2)$ при $x = 0.7$ (логарифмическая шкала по оси Y)

На графике видно монотонное убывание погрешности с ростом числа членов ряда. На логарифмической шкале это выглядит как почти линейное снижение ошибки, что соответствует ожидаемому поведению остатка ряда Тейлора.

Выводы

В ходе выполнения лабораторной работы №1 были получены следующие результаты и сделаны выводы:

- **Сравнение float64 и float32.** При уменьшении x до значений порядка 10^{-4} и ниже точность float32 резко падает: относительная ошибка достигает 1, появляются нулевые значения и NaN. Это вызвано катастрофической потерей значащих цифр при вычитании близких чисел в числителе $x - \sin x$ и делении на очень малое x^3. Float64 сохраняет приемлемую точность до $x \sim 10^{-7}$–10^{-8}, после чего также начинает деградировать.

- **Машинный эпсилон.** После исправления логической ошибки в коде получены корректные значения: $\varepsilon_{64} \approx 2.22 \cdot 10^{-16}$ и $\varepsilon_{32} \approx 1.19 \cdot 10^{-7}$. Результаты подтверждают теоретические оценки стандарта IEEE 754 и показывают, как разрядность напрямую влияет на минимальную достижимую погрешность вычислений.

- **Ряд Тейлора для** $\ln(1-x^2)$**.** Ряд сходится к точному значению, но скорость сходимости зависит от x. При $x = \pm 0.6$ сходимость достаточно быстрая (12–20 членов для $\varepsilon = 10^{-10}$), а при $x = 0.8$, близком к границе $|x| < 1$, требуется существенно больше членов (44 для $\varepsilon = 10^{-10}$). Это согласуется с теорией сходимости степенных рядов.

- **График погрешности.** Построенная зависимость подтверждает, что ошибка убывает с ростом числа слагаемых. На логарифмической шкале снижение ошибки близко к линейному, что типично для асимптотического поведения остатка ряда.

- **Источники погрешностей.** Основные источники ошибок: округление в машинной арифметике, вычитание близких чисел (catastrophic cancellation) и усечение ряда. Для повышения точности рекомендуется использовать более высокую разрядность, устойчивые формулы и достаточное число членов ряда, особенно вблизи границ области сходимости.

Таким образом, цель работы достигнута: исследованы погрешности машинных вычислений, продемонстрированы различия между float32 и float64, изучено поведение ряда Тейлора и построены соответствующие графики.

Ответы на контрольные вопросы

1. **Что такое абсолютная и относительная погрешности?**
   Абсолютная погрешность — модуль разности точного и приближённого значений: $\Delta x = |x_{\text{точн}} - x_{\text{приб}}|$. Относительная погрешность — отношение абсолютной погрешности к модулю точного значения (при $x_{\text{точн}} \neq 0$): $\delta = \Delta x / |x_{\text{точн}}|$.

2. **Почему выражение** $\frac{x-\sin x}{x^3}$ **при малых** x **считается неустойчивым?**
   При малых x значения x и $\sin x$ очень близки, их разность $x - \sin x \approx x^3/6$ — величина на несколько порядков меньше исходных чисел. При вычитании близких чисел происходит катастрофическая потеря значащих цифр, и итоговая точность результата резко падает. Деление на x^3 лишь усиливает эффект.

3. **Как экспериментально определить машинный эпсилон?**
   Нужно итеративно делить $\varepsilon = 1$ пополам, пока $1+\varepsilon$ в заданной разрядности не станет равным 1. Первое значение ε, при котором $1+\varepsilon > 1$, умноженное на 2, даёт машинный эпсилон. Важно не прерывать цикл досрочно (как в ошибочной реализации).

4. **От чего зависит скорость сходимости ряда Тейлора?**
   Скорость сходимости зависит от точки разложения и расстояния до ближайшей особой точки функции. Для $\ln(1 - x^2)$ особые точки находятся при $x = \pm 1$. Чем ближе x к этим точкам, тем медленнее сходится ряд и тем больше членов требуется для заданной точности.

5. **Какие способы повышения точности вычислений можно применить?**

   - Использовать более высокую разрядность (float64 вместо float32).
   - Переформулировать выражения, чтобы избежать вычитания близких чисел.
   - Применять устойчивые алгоритмы и библиотечные функции.
   - Суммировать ряды в порядке возрастания модулей слагаемых.
   - Учитывать область сходимости и выбирать подходящие методы аппроксимации.